

A Smart Parking System with Vehicle Detection and Barrier Control

Nguyen Sy Minh Man, Do Thanh Triet, Tran Dang Khoa, and Bui Dinh Long FPT University, Ho Chi Minh Campus, Vietnam
 {syminhman01, thanhtriet1007, khoavainoi2398, buidinhlongwork}@gmail.com

Abstract—This paper presents the design, implementation, and experimental evaluation of a small-scale smart parking system that combines ultrasonic vehicle-presence sensing, ESP32-CAM image capture, server-side vehicle detection, Arduino-based barrier control, database storage, and web-based monitoring. The system uses two sensor-camera modules for the entry and exit gates and an Arduino Master to coordinate local barrier control. When an object is detected near a gate, an ESP32-CAM captures an image and sends it to a backend server, where a YOLO26l model determines whether an allowed vehicle class is present. The resulting decision is propagated to the control system using a fail-close strategy.

The final YOLO26l experiment consisted of 146 confirmed positive images of physical vehicle models captured by the implemented ESP32-CAM modules. The model produced 66 OPEN decisions and 80 CLOSE decisions, corresponding to a positive-sample detection rate of 45.21% and a miss rate of 54.79%. The mean server-side inference time was 432.23 ms. A practical comparison using an earlier vehicle-model image source showed that YOLO26l achieved a higher positive-sample detection rate than the historical YOLOv8n run, 94.23% compared with 80.43%, but required approximately 4.51 times longer inference time. Because the experiment contained no verified negative samples and the model-comparison runs were not paired one-to-one, the reported results do not represent complete classification accuracy or a controlled benchmark. The prototype demonstrates the feasibility and limitations of integrating low-cost IoT hardware with backend vehicle detection and centralized parking monitoring.

Index Terms—Internet of Things, smart parking, ESP32-CAM, Arduino UNO, ultrasonic sensor, YOLO, vehicle detection, barrier control, web dashboard.

I. INTRODUCTION

A. Background and Problem

The increasing number of vehicles has created a growing demand for efficient parking management in campuses, offices, residential buildings, and other small-to medium-sized facilities. Conventional parking systems commonly depend on manual gate operation and human-based vehicle recording, which may result in slow vehi-

cle processing, incomplete entry and exit records, and limited visibility of current parking activity. Internet of Things (IoT)-based parking systems have therefore been developed to automate vehicle detection, collect parking information, and provide centralized monitoring through web or mobile applications [1], [2].

Low-cost smart parking prototypes frequently use ultrasonic sensors because they are inexpensive, simple to integrate, and capable of detecting the presence of an object near a parking gate or parking space [2], [3], [4]. However, distance sensing alone cannot determine whether the detected object is a valid vehicle. A person, wall, or another nearby object may produce a similar trigger. Consequently, a sensor-only system cannot provide reliable visual evidence for individual parking events.

Camera-based systems provide an additional vehicle-verification layer by capturing images that can be processed and stored for later review. ESP32-CAM modules and conventional web cameras have been used in parking prototypes to support vehicle monitoring, contactless access, and future license-plate recognition [5], [6]. Deep-learning detectors such as YOLO can further distinguish allowed vehicle classes from people and other objects [7]. Nevertheless, image-based verification introduces additional processing time, communication overhead, and dependence on image quality, lighting, camera position, and server availability.

This project addresses these issues through a small-scale smart parking prototype that integrates ultrasonic presence sensing, ESP32-CAM image capture, server-side YOLO vehicle detection, Arduino-based barrier control, database storage, and web-based monitoring. The implemented architecture contains one Arduino Uno R3 Master, two Arduino Uno R3 Slave modules for the ENTRY and EXIT directions, two HC-SR04 ultrasonic sensors, two ESP32-CAM modules, and an SG90 servo barrier. Captured images are transmitted through Wi-Fi to a local FastAPI server, where YOLO26l generates an OPEN or CLOSE decision. The decision is then propagated to the embedded control layer. When a valid result is not obtained, the system follows a fail-close

policy and keeps the barrier closed.

B. Objective and Contributions

The objective of this study is to design, implement, and evaluate a low-cost IoT parking prototype that combines object-triggered image capture with vehicle-class verification and centralized event monitoring. The main contributions are as follows:

- 1) A modular ENTRY-EXIT architecture that separates vehicle sensing, image capture, local barrier control, server-side detection, and dashboard monitoring.
- 2) A two-stage decision process in which ultrasonic sensing initiates image capture and YOLO-based verification determines whether the barrier may be opened.
- 3) An empirical evaluation of YOLO26l using real ESP32-CAM captures of physical vehicle models, together with a practical comparison against a historical YOLOv8n run.

The study is limited to a laboratory-scale prototype using physical vehicle models. It does not evaluate full-size outdoor vehicles, production barrier safety, complete classification accuracy, or long-duration deployment reliability.

C. Research Questions

The evaluation is guided by the following research questions:

RQ1: What positive-sample vehicle-detection rate does YOLO26l achieve on the selected ESP32-CAM vehicle-model dataset?

RQ2: What detection-rate and server-side inference-time trade-off is observed between YOLO26l and the historical YOLOv8n run on the available earlier test source?

RQ1 is evaluated using the number of OPEN decisions divided by the total number of confirmed positive vehicle images. This metric is reported as a positive-sample detection rate rather than overall classification accuracy because the dataset does not contain verified negative samples.

RQ2 compares the observed positive-sample detection rate and mean server-side inference time of the two model runs. Because the available YOLOv8n and YOLO26l results do not contain one-to-one matched test-case identifiers, the comparison is treated as a same-source practical comparison rather than a controlled paired benchmark.

II. RELATED WORK AND DESIGN RATIONALE

Existing smart parking systems can be broadly divided into sensor-based, camera-based, and hybrid architectures. Sensor-based systems commonly use ultrasonic or infrared sensors to detect vehicle presence and update parking availability [2], [3], [4]. These solutions are appropriate for low-cost prototypes because the sensors are inexpensive, require limited computation, and can provide fast object-presence information. However, their measurements indicate only that an object is located within a defined distance. They cannot independently verify whether the detected object is a vehicle.

Camera-based systems address this limitation by providing visual evidence for each parking event. Previous implementations have used ESP32-CAM modules or web cameras for contactless parking control, vehicle monitoring, and license-plate processing [5], [6]. More recent approaches apply deep-learning detectors such as YOLO to identify vehicles from captured images [7]. Although computer vision provides stronger vehicle-verification capability than distance sensing alone, it also requires higher computational resources and depends on camera quality, lighting, object scale, network communication, and inference time.

Web and mobile monitoring have also been incorporated into IoT parking systems to present parking availability, vehicle events, and system status to administrators [1], [2]. These systems demonstrate the value of centralized monitoring, but several reviewed studies focus mainly on parking-slot availability or image processing without fully integrating sensing, image capture, barrier actuation, event storage, and dashboard monitoring in one low-cost gate-control prototype.

Based on these trade-offs, the proposed system uses a hybrid, two-stage detection process. The ultrasonic sensor acts as the initial trigger, while the ESP32-CAM and backend YOLO detector provide a second vehicle-verification layer. This structure avoids continuous image upload and inference when no nearby object is detected, while retaining visual evidence for triggered parking events.

The use of Arduino Slave modules instead of assigning all functions to the ESP32-CAM is an architectural decision rather than a claim that the Arduino Uno has greater processing capability. The ESP32-CAM modules are dedicated to image capture and Wi-Fi communication, whereas the Arduino modules maintain local sensor timing and barrier-control coordination. This separation makes the ENTRY and EXIT modules easier to test independently and prevents camera or network processing from directly controlling the servo.

TABLE I
DESIGN DECISIONS FOR THE PROPOSED SMART PARKING SYSTEM

Design Layer	Alternatives Considered	Selected Design and Rationale
Presence detection	Ultrasonic sensor, infrared sensor, or camera-only detection	HC-SR04 ultrasonic sensors are used as low-cost event triggers. They provide simple distance-based detection before the system starts image capture and backend inference.
Image acquisition	Fixed web camera, smartphone camera, or ESP32-CAM	Two ESP32-CAM modules are assigned to the ENTRY and EXIT directions. Their integrated camera and Wi-Fi functions support distributed image capture without requiring a continuously connected computer at each gate.
Local controller	Single ESP32 controller or modular Arduino controllers	One Arduino Uno R3 Master and two Arduino Uno R3 Slaves are used. The Slaves handle direction-specific sensing and camera coordination, while the Master owns the barrier decision and servo command. This separates local control from image and network processing.
AI processing	On-device detection or server-side inference	YOLO26l inference is executed on a local FastAPI server because the ESP32-CAM is used primarily for image capture and transmission and does not provide sufficient resources for the selected YOLO model.
Communication	Single shared controller, I2C, UART, MQTT, or HTTP	I2C is used between the Arduino Master and Slave modules, serial communication is used for local Slave-camera coordination, and HTTP is used for image upload and server response. Each protocol is assigned according to the corresponding communication scope.
Monitoring	Local LCD only or centralized web monitoring	A database and Java-based web dashboard store and display detection events, gate states, images, and parking occupancy. The LCD1602 is used as a local debugging and status display rather than the primary user-facing interface.

The modular design also introduces disadvantages. It requires more boards, wiring, power connections, and communication interfaces than a single-controller architecture. In addition, the current Master implementation depends on responses from both Slave modules; therefore, an inactive Slave can interrupt normal system operation. The selected architecture is consequently suitable for demonstrating modular integration and control ownership in the prototype, but it should not be interpreted as the minimum-cost or most fault-tolerant architecture for a production parking system.

III. PROPOSED SYSTEM AND IMPLEMENTATION

A. System Architecture and Hardware

The proposed smart parking prototype follows a modular architecture consisting of three main layers: the local control layer, the sensor-camera layer, and the server-monitoring layer. The overall architecture is shown in Fig. 1.

The local control layer contains one Arduino Uno R3 Master, an SG90 servo motor, and an LCD1602 display. The Master acts as the owner of the final local barrier command. It polls the ENTRY and EXIT Slave modules, receives their pending detection decisions, updates the local status display, and commands the servo to open or close the prototype barrier. The LCD1602 is used primarily as a debugging and local status display rather than as the main user-facing interface.

Insert image: blockDiagram.png

Fig. 1. Block diagram of the proposed smart parking system.

The sensor-camera layer contains two independent modules corresponding to the ENTRY and EXIT directions. Each module includes one Arduino Uno R3 Slave, one HC-SR04 ultrasonic sensor, and one ESP32-CAM. The ENTRY Slave uses I2C address 0×08 , whereas the EXIT Slave uses 0×09 . The ultrasonic sensor detects whether an object is located within the configured trigger range. The corresponding ESP32-CAM then captures and uploads an image for server-side vehicle verification.

The server-monitoring layer contains a FastAPI detection service, a YOLO26l model, an SQL database, and a Java-based web dashboard. The server receives JPEG images from the ESP32-CAM, performs vehicle detection, and returns an OPEN or CLOSE decision. Detection metadata and image paths are stored in the database for dashboard monitoring and later review.

Table II summarizes the principal hardware and soft-

TABLE II
IMPLEMENTED SYSTEM CONFIGURATION

Component	Implemented Configuration
Master controller	Arduino Uno R3 controlling the servo, local display, and Slave polling process
Sensor modules	Two Arduino Uno R3 Slaves with HC-SR04 ultrasonic sensors for ENTRY and EXIT
Camera modules	Two ESP32-CAM modules with Wi-Fi image upload capability
Barrier actuator	One SG90 servo motor representing the parking barrier
Camera configuration	VGA frame size, JPEG quality 12, one frame buffer, and 20 MHz XCLK
Detection server	FastAPI service using the <code>yolo261.pt</code> model with an input size of 640 pixels
Detection policy	Confidence threshold of 0.45 with allowed vehicle classes <code>car</code> , <code>truck</code> , <code>bus</code> , and <code>motorcycle</code>
Monitoring layer	SQL database and Java web dashboard for occupancy, gate state, detection events, and captured images

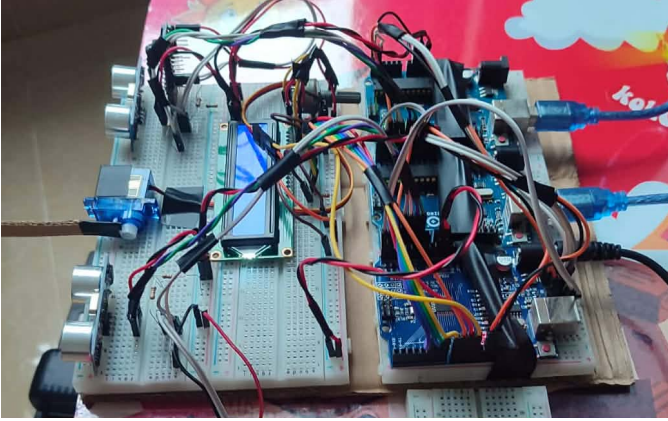


Fig. 2. Complete hardware prototype of the proposed smart parking system.

ware configuration used in the implemented prototype.

The complete integrated prototype is shown in Fig. 2. All sensing, communication, camera, and barrier-control components are wired as one assembly. Therefore, separate photographs of individual subsystems are not required.

B. Communication and Control Workflow

The system uses different communication mechanisms according to the responsibility of each module. I2C is used between the Arduino Master and the two Arduino Slaves. Local serial communication is used between each

Slave and its associated ESP32-CAM. HTTP over Wi-Fi is used between the ESP32-CAM and the FastAPI server.

The complete operating sequence is described as follows:

- 1) The HC-SR04 sensor continuously checks the distance in front of the corresponding ENTRY or EXIT gate.
- 2) When an object satisfies the configured detection condition, the Arduino Slave sends a capture request to its ESP32-CAM.
- 3) The ESP32-CAM captures a JPEG image and uploads it to the FastAPI detection endpoint through Wi-Fi.
- 4) The server executes YOLO261 inference and checks whether the best accepted detection belongs to one of the allowed vehicle classes and satisfies the confidence threshold.
- 5) If a valid allowed vehicle is detected, the server returns OPEN. Otherwise, it returns CLOSE.
- 6) The ESP32-CAM forwards the result to the Arduino Slave. The Slave stores the decision until it is retrieved during the next Master polling cycle.
- 7) The Arduino Master polls the relevant Slave through I2C and obtains the pending decision.
- 8) The Master opens the servo barrier only when a valid OPEN result is available. When the result is missing, invalid, expired, or explicitly CLOSE, the barrier remains closed.

This workflow implements a fail-close control policy. A sensor trigger alone is not sufficient to open the barrier. The Master requires a valid positive decision from the camera and server processing chain. Therefore, an image-upload failure, Wi-Fi interruption, HTTP timeout, invalid server response, or missing Slave result does not directly activate the barrier.

The Master polls the Slave modules at an interval of approximately 1000 ms. The Slave, camera, and Master use separate timeout windows to prevent a request from remaining active indefinitely. These timeout values are configuration limits rather than measured response-time results and are therefore not treated as experimental latency.

C. Backend, Database, and Dashboard

The backend server receives an image and direction identifier from the ESP32-CAM. The YOLO inference service processes the image and generates a list of detections containing the predicted class, confidence score, and bounding-box information. A detection is accepted only when its class belongs to the configured vehicle-class list and its confidence is greater than or equal to 0.45.

The server stores each detection event with information such as the gate direction, timestamp, decision, confidence, predicted class, raw image path, and annotated image path. The core database structure contains the following records:

- `parking_occupancy`, which stores the parking capacity and current number of vehicles inside;
- `gates`, which stores the current ENTRY and EXIT gate states;
- `detection_events`, which stores the result and image information for each server request.

The Java web dashboard retrieves these records through controller, DAO, and model classes. The overview interface displays the number of vehicles currently inside, daily ENTRY and EXIT counts, gate status, camera and sensor status, and the latest ENTRY and EXIT images. Additional event-history information can be presented through a parking-log page.

The dashboard is used for centralized observation and record review. It does not directly replace the embedded fail-close control logic. The Arduino Master remains responsible for the physical servo command, whereas the server and dashboard provide vehicle verification, event storage, occupancy updates, and administrative monitoring.

The implemented architecture therefore separates four responsibilities: sensor triggering, image acquisition, AI verification, and physical barrier control. This separation supports modular testing and clear control ownership, although it also increases the number of boards, communication links, and possible failure points compared with a single-controller design.

IV. EXPERIMENTAL SETUP

A. Evaluation Scope

The experimental evaluation focused on the server-side vehicle-detection component because the available archive contains per-request YOLO logs but does not contain synchronized timestamps for the complete sensor-to-servo workflow. The evaluation therefore reports positive-sample detection rate, miss rate, accepted-detection confidence, and server-side inference time.

The experiment did not measure complete classification accuracy because the selected dataset contains confirmed positive vehicle images only. It also did not measure ultrasonic-sensor accuracy, dashboard update delay, system uptime, or end-to-end barrier response time. These limitations are explicitly separated from the measured results.

B. Dataset and Log Selection

The available archive contained 421 valid JSON result files collected between July 13 and July 17, 2026. All files were readable, used one consistent schema, and contained unique request identifiers. The archive included final experiments, historical runs, development tests, and later verification captures.

The main YOLO26l dataset was reconstructed as the unique continuous sequence satisfying all of the following conditions:

- 146 result logs;
- 95 ENTRY requests and 51 EXIT requests;
- 66 OPEN decisions and 80 CLOSE decisions;
- model file `yolo26l.pt`;
- collection period from July 14, 2026 at 23:04:50 to July 15, 2026 at 00:44:00.

All 146 images were treated as positive samples because each image was captured from a test case containing a physical vehicle model. An OPEN decision therefore represents a successful accepted vehicle detection, whereas a CLOSE decision represents a miss for this positive-only dataset.

Two additional groups were used for the practical model comparison. The historical YOLOv8n group contained 46 EXIT logs collected on July 13, 2026. The later YOLO26l comparison group contained 52 logs collected on July 15, 2026 from the available earlier vehicle-model test source. The two groups were not paired one-to-one.

C. Detection Configuration

The server used the following principal inference configuration:

- YOLO26l model file: `yolo26l.pt`;
- comparison baseline: `yolov8n.pt`;
- input image size: 640 pixels;
- confidence threshold: 0.45;
- accepted classes: `car`, `truck`, `bus`, and `motorcycle`.

The ESP32-CAM modules used VGA frame size, JPEG quality 12, one frame buffer, and a 20 MHz XCLK. These values are implementation configuration parameters rather than experimental performance results.

D. Evaluation Metrics

For the positive-only dataset, the positive-sample detection rate was calculated as

$$\text{Detection Rate} = \frac{N_{\text{OPEN}}}{N_{\text{positive samples}}} \times 100\%. \quad (1)$$

The miss rate was calculated as

TABLE III
MAIN YOLO26L RESULTS ON 146 POSITIVE SAMPLES

Metric	Result
Total samples	146
ENTRY / EXIT	95 / 51
OPEN / CLOSE	66 / 80
Positive-sample detection rate	45.21%
Miss rate	54.79%
Mean inference time	432.23 ms
Median inference time	415.76 ms
Minimum / maximum inference	386.61 / 837.82 ms
Inference standard deviation	62.16 ms
95th-percentile inference	505.17 ms
Mean accepted confidence	72.63%

$$\text{Miss Rate} = \frac{N_{\text{CLOSE}}}{N_{\text{positive samples}}} \times 100\%. \quad (2)$$

Server-side inference performance was summarized using the mean, median, minimum, maximum, standard deviation, and 95th percentile of the logged `inference_ms` values. Accepted-detection confidence was summarized only for requests that contained an allowed vehicle detection.

For RQ2, the observed trade-off was represented by the difference in positive-sample detection rate and the ratio between the two mean inference times. Because the model runs contained different request counts and lacked shared test-case identifiers or source-image hashes, no paired significance test was performed.

E. Communication Observation Scope

Master-Slave communication behavior was evaluated through integration observation rather than through the JSON archive. Communication succeeded during the observed tests when both Slave modules were active and responsive. The current Master implementation stopped normal operation when either one or both Slaves became inactive. Since the number of polling attempts was not logged, this observation is not reported as a numerical communication-success rate.

V. RESULTS AND DISCUSSION

A. Main YOLO26l Results

Table III summarizes the main experiment. YOLO26l returned 66 OPEN decisions from 146 confirmed positive vehicle-model images. The resulting positive-sample detection rate was 45.21%, while the miss rate was 54.79%.

The ENTRY subset produced 41 successful detections from 95 requests, corresponding to 43.16%. The EXIT

TABLE IV
SAME-SOURCE PRACTICAL MODEL COMPARISON

Run	Samples	Detection Rate	Mean Inference
YOLOv8n	46	80.43%	93.42 ms
YOLO26l	52	94.23%	421.39 ms

subset produced 25 successful detections from 51 requests, corresponding to 49.02%. Although the EXIT rate was 5.86 percentage points higher, this difference does not demonstrate that the EXIT camera was more accurate. The two directions had different sample counts, and the archive did not provide controlled labels for lighting, distance, camera angle, or identical test cases.

Among the 66 accepted detections, 43 were predicted as `truck` and 23 as `car`. No accepted `bus` or `motorcycle` predictions occurred in the main dataset. These values represent model output labels rather than class-level accuracy because no manually verified class ground truth was included.

Of the 80 CLOSE results, 34 still contained at least one detected object outside the accepted vehicle-class list, while 46 contained no detection above the threshold. Common non-accepted labels included `person`, `bed`, `mouse`, `cat`, `chair`, and `remote`. This behavior indicates that some vehicle models or background regions were misinterpreted as unrelated classes. However, the exact causes cannot be isolated without a matched image-level annotation set.

The measured 45.21% therefore answers RQ1 as a positive-sample detection rate under the implemented prototype conditions. It must not be interpreted as overall accuracy. The high miss rate shows that the current low-resolution vehicle-model workflow remains sensitive to camera placement, object scale, lighting, background content, and the selected confidence threshold.

B. YOLOv8n–YOLO26l Practical Comparison

Table IV presents the available practical comparison. The historical YOLOv8n run produced 37 OPEN decisions from 46 positive requests, corresponding to 80.43%. The YOLO26l comparison run produced 49 OPEN decisions from 52 requests, corresponding to 94.23%.

The observed positive-sample detection rate increased by 13.80 percentage points. In contrast, mean inference time increased by 327.97 ms. YOLO26l required approximately 4.51 times the mean server-side inference time of YOLOv8n.

These results answer RQ2 by showing a clear detection-rate and inference-time trade-off. YOLO26l

produced more usable detections on the available earlier vehicle-model test source, but at substantially higher computational cost. The result supports selecting YOLO26l when detection usefulness is prioritized over server inference speed in the current laboratory prototype.

The comparison is not a controlled paired benchmark. YOLOv8n and YOLO26l were represented by 46 and 52 logs respectively, and the archive did not contain a common `test_case_id`, image hash, or one-to-one mapping between results. The observed difference may therefore also reflect differences in capture direction, image selection, or test conditions. The conclusion is limited to a same-source practical comparison and does not establish universal superiority of YOLO26l.

C. Integration Observations

During integration tests, the Master communicated successfully while both ENTRY and EXIT Slave modules were active and responsive. When either Slave became inactive, the current implementation stopped normal operation. This behavior confirms the functional dependency between the Master and both Slave modules, but it also exposes a fault-tolerance limitation.

The fail-close policy reduced the risk of unintended barrier opening. A missing or invalid camera result, failed Wi-Fi transfer, HTTP timeout, or unavailable server decision resulted in the barrier remaining closed. This behavior was observed functionally, but the archive did not record enough end-to-end success and failure counts to calculate a complete workflow-success rate.

Power stability was also an important implementation factor. The ESP32-CAM modules operated more reliably when powered through a stable regulated 5 V source than through the earlier unstable power path. This observation supports separating camera power considerations from the 12 V supply path used elsewhere in the prototype.

D. Validity and Limitations

The current evidence has several limitations:

- 1) The main dataset contains only positive vehicle samples. Precision, specificity, false-positive rate, complete accuracy, and F1-score cannot be calculated.
- 2) No independent class ground truth was provided. Predicted `car` and `truck` labels cannot be treated as verified class accuracy.
- 3) The log archive contains final, historical, development, and verification runs without an explicit `experiment_id`.

- 4) Ultrasonic distance, trigger state, and sequence identifiers were not recorded in the selected JSON logs.
- 5) End-to-end gate response time was not measured with synchronized timestamps across the sensor, Slave, ESP32-CAM, server, Master, and servo.
- 6) The two model runs were not paired one-to-one.
- 7) The prototype used physical vehicle models and an SG90 servo. The results do not represent full-size outdoor vehicles or production barrier safety.
- 8) Long-duration operation, unstable Wi-Fi, broad lighting variation, network congestion, and simultaneous ENTRY-EXIT events were not evaluated quantitatively.

End-to-end gate response time is therefore intentionally excluded from the reported performance results. The mean value of 432.23 ms refers only to server-side YOLO26l inference and not to the complete sensor-to-barrier workflow.

VI. CONCLUSION

This study designed and implemented a small-scale smart parking prototype that combines ultrasonic sensing, ESP32-CAM image capture, server-side YOLO vehicle detection, Arduino-based barrier control, database storage, and web monitoring. The modular architecture assigns sensor and camera coordination to the ENTRY and EXIT Slave modules, while the Arduino Master retains ownership of the local servo command. A fail-close policy prevents a sensor trigger or invalid server result from opening the barrier directly.

For RQ1, YOLO26l produced 66 OPEN decisions from 146 confirmed positive ESP32-CAM vehicle-model images. The corresponding positive-sample detection rate was 45.21%, with a miss rate of 54.79%. The mean server-side inference time was 432.23 ms. These results describe performance on a positive-only laboratory dataset and do not represent complete classification accuracy.

For RQ2, the available earlier-source comparison showed that YOLO26l increased the observed positive-sample detection rate from 80.43% for the historical YOLOv8n run to 94.23%. However, mean inference time increased from 93.42 ms to 421.39 ms, making YOLO26l approximately 4.51 times slower. This result demonstrates a practical trade-off between usable detections and server inference speed, but it is not a controlled paired benchmark.

The prototype demonstrates that low-cost sensing, distributed image capture, backend AI verification, and centralized monitoring can be integrated into one functional smart parking workflow. However, the high miss

rate in the main dataset, the dependency on both Slave modules, the absence of synchronized end-to-end timing, and the lack of negative samples limit the strength of the current evaluation.

Future work should introduce an experiment manifest, shared image hashes, paired model tests, manually verified positive and negative ground truth, and synchronized timestamps from sensor trigger to servo movement. Additional evaluation should cover lighting variation, camera distance, full-size vehicles, network interruption, long-duration operation, and independent recovery when one Slave module becomes unavailable.

REFERENCES

- [1] A. Z. M. T. Kabir, A. M. Mizan, P. K. Saha, M. S. Hasan, and M. Pramanik, "An iot based intelligent parking system for the unutilized parking area with real-time monitoring using mobile and web application," *Asian Journal of Convergence in Technology*, vol. 7, no. 2, pp. 107–113, 2021.
- [2] I. Jung, "An iot-based smart parking management system," *International Journal of Computational Vision and Robotics*, vol. 10, no. 2, pp. 122–132, 2020.
- [3] S. Y. C. Hong, C. C. Kang, J. D. Tan, and M. Ariannejad, "Smart parking system using iot sensors," *Journal of Engineering Technology and Applied Physics*, vol. 5, no. 1, pp. 63–69, 2023.
- [4] Y. Allbadi, J. N. Shehab, and M. M. Jasim, "The smart parking system using ultrasonic control sensors," *IOP Conference Series: Materials Science and Engineering*, vol. 1076, no. 1, p. 012064, 2021.
- [5] V. A. Kusuma, H. Arof, S. S. Suprpto, B. Suharto, R. A. Sinulingga, and F. Ama, "An internet of things-based touchless parking system using esp32-cam," *International Journal of Reconfigurable and Embedded Systems*, vol. 12, no. 3, pp. 329–335, 2023.
- [6] E. Saputra, N. P. IS, E. Fitriani, T. Ariyadi, and A. H. A. Tarecha, "Car parking monitoring system based on location plate detection using web camera," *Circuit: Jurnal Ilmiah Pendidikan Teknik Elektro*, vol. 10, no. 1, pp. 25–39, 2026.
- [7] G. P. C. P. da Luz, G. M. Sato, L. F. G. Gonzalez, and J. F. Borin, "Smart parking with pixel-wise roi selection for vehicle detection using yolov8, yolov9, yolov10, and yolov11," *arXiv preprint arXiv:2412.01983*, 2024.